

ABSTRACT

Recommender systems are essential in modern platforms like e-commerce and movie streaming, helping users navigate large numbers of choices. It help users discover relevant content from large item collections. Traditional collaborative filtering methods rely mainly on user–item interactions and often struggle to incorporate richer domain knowledge. This project develops a graph-based movie recommendation system using the MovieLens 20M dataset, which includes over 20 million ratings across more than 27,000 movies and 138,000 users.

Four models were evaluated: Item-Based Collaborative Filtering (ItemKNN), Matrix Factorization with Bayesian Personalized Ranking (MF-BPR), LightGCN, and GraphSAGE. The dataset was cleaned, encoded, and transformed into a graph structure including users, movies, genres, and tags. Baseline models used user–movie interactions, while graph models leveraged graph structure and additional domain relationships.

Performance was evaluated using ranking metrics such as Precision@K, Recall@K, NDCG@K, and HitRate@K. Results show that ItemKNN achieved the best performance, followed by MF-BPR, LightGCN, and GraphSAGE. Although GraphSAGE achieved lower ranking scores, it enabled richer representation by incorporating domain knowledge. Overall, the study highlights that classical collaborative filtering remains highly competitive, while graph-based approaches provide a flexible framework for integrating complex relationships in recommendation systems.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Recommendation systems have become essential in modern digital services such as e-commerce, media streaming, online learning, and social platforms. Their primary role is to reduce information overload by identifying items that are likely to match a user's preferences. In movie recommendation, this means predicting which unseen movies a user is most likely to enjoy based on historical interaction patterns and, potentially, content-related information.

Traditional recommendation methods have achieved strong results, especially collaborative filtering approaches based on neighborhood similarity and latent factor models. Item-based collaborative filtering became a prominent technique because it computes similarities directly between items and can generate effective recommendations from historical interaction patterns. Matrix factorization(MF-BPR) methods later became highly influential because they learn compact latent representations of users and items, allowing hidden preference structure to be captured efficiently.

However, recommendation data is often sparse. Even in benchmark datasets, the number of observed interactions is far smaller than the number of possible user–item combinations. The MovieLens 20M dataset, for example, contains 20,000,263 ratings and 465,564 tag applications across 27,278 movies by 138,493 users, which implies a highly sparse interaction matrix. This sparsity limits the effectiveness of methods that depend purely on interaction frequency.

Recent research has shown that graphs provide a natural representation for recommendation data because users and items can be modeled as nodes, while their interactions can be modeled as edges. Graph neural networks extend this

representation by learning from local and multi-hop relationships in the graph. Light Graph Convolutional Network(LightGCN) was proposed as a simplified graph convolution architecture specifically for recommendation, focusing only on neighborhood aggregation in the user–item graph. GraphSAGE introduced an inductive graph representation learning framework based on neighborhood sampling and aggregation, and it is flexible enough to support richer graph structures with multiple node types.

This project investigates whether graph-based learning can provide a richer and more flexible recommendation framework than classical collaborative filtering. In particular, it explores not only a collaborative user–movie graph, but also a heterogeneous graph that incorporates domain-specific knowledge through genres and tags. The project also compares graph-based and non-graph-based methods under a shared evaluation pipeline and demonstrates their real-time behavior through a deployment interface.

1.2 PROBLEM STATEMENT

Movie recommendation is a ranking problem in which the system must identify the most relevant unseen movies for each user. The problem becomes challenging because user feedback is incomplete and sparse: most users interact with only a small subset of available movies. As a result, the recommendation model must infer preferences from limited evidence.

Traditional collaborative filtering methods are effective but face several limitations. First, they rely mainly on user–movie interactions and may not directly incorporate additional semantic information such as genres or tags. Second, they can struggle when co-occurrence information is weak or when user histories are limited. Third, they often model preference from only one relational viewpoint, whereas recommendation data naturally contains multiple interconnected entities.

The central problem addressed in this thesis is therefore:

How can a graph-based recommendation system integrate collaborative signals and domain-specific knowledge to model movie recommendations more expressively, and how does its performance compare with strong baseline models such as ItemKNN and Matrix Factorization?

This problem is addressed through the design, implementation, evaluation, and deployment of a multi-model recommendation pipeline.

1.3 PROPOSED APPROACH

The proposed approach is a graph-based movie recommendation system that combines traditional collaborative filtering and graph neural network methods in a unified experimental framework.

First, the MovieLens dataset is cleaned, validated, and transformed into model-ready data. Second, recommendation relationships are modeled both in tabular form and in graph form. Third, four different recommendation models are trained and compared:

- Item-Based Collaborative Filtering (ItemKNN)
- Matrix Factorization with BPR
- Light Graph Convolutional Network (LightGCN)
- GraphSAGE

The baseline models operate mainly on user–movie interaction data. The graph models, however, use graph-structured representations, and GraphSAGE extends the graph with domain-specific nodes such as genres and tags.

The overall workflow is:

Raw Data → Preprocessing → Graph Construction → Model Training →
Evaluation → Model Hosting → Streamlit Deployment

This approach enables both quantitative comparison and practical demonstration.

CHAPTER 2: DATASET

2.1 DATASET OVERVIEW

The dataset used in this project is the MovieLens 20M Dataset, which is a widely used benchmark dataset for recommender system research. The dataset was collected and released by the GroupLens Research at the University of Minnesota.

The MovieLens dataset family has been extensively used in recommendation system research because it provides high-quality user–item interaction data along with additional metadata such as movie genres and user-generated tags. These datasets enable researchers to develop, evaluate, and compare different recommendation algorithms under standardized conditions.

The MovieLens 20M dataset contains approximately 20 million movie ratings provided by real users over several years. In addition to ratings, the dataset includes metadata about movies and textual tags created by users to describe movies.

2.2 DATASET STATISTICS

The MovieLens 20M dataset contains the following overall statistics:

ATTRIBUTE	VALUE
Total Ratings	20,000,263
Total Tag Applications	465,564
Total Movies	27,278
Total Users	138,493

Rating Scale	0.5 - 5.0 Stars
Rating Increments	0.5
Dataset Timespan	January 1995 - March 2015

Key characteristics:

- The dataset contains dense rating signals compared to many recommendation datasets.
- Each user has rated at least 20 movies.
- The ratings are explicit feedback, meaning users explicitly provided rating scores rather than implicit interactions such as clicks or views.
- The dataset also includes user-generated tags, which provide additional semantic information about movies.

Because of these properties, MovieLens 20M is considered one of the most reliable datasets for evaluating recommender systems.

2.3 FILES USED IN THE DATASET

The dataset contains several CSV files. In this project, the following three files were used:

2.3.1 Ratings.csv

This file contains the main user–movie interaction data.

Each row represents a rating given by a user to a movie.

Columns:

COLUMN	DESCRIPTION
userID	Unique Identifier for a user
movieID	Unique Identifier for a movie
Rating	Rating given by the user (0.5 - 5.0)
Timestamp	Time when the rating was recorded

Example:

userID	movieID	Rating	Timestamp
1	296	3.5	1112486027
1	318	4.0	1231234567
2	593	5.0	835355432

This file contains 20,000,263 ratings, which form the primary interaction data used by recommendation models.

2.3.2 Movies.csv

This file provides metadata about movies

Columns:

COLUMN	DESCRIPTION
movieID	Unique Identifier for the movie.
Title	Movie title including release year.
Genres	List of genres associated with the movie.

Genres are stored as a pipe-separated string.

Example:

Action|Adventure|Sci-Fi

Comedy|Romance

Drama

Example rows:

movieID	Title	Genres
1	Toy Story (1995)	Adventure
2	Jumanji (1995)	Adventure
3	Grumpier Old Men (1995)	Comedy

This file contains 27,278 movies.

2.3.3. Tags.csv

This file contains user-generated descriptive tags for movies.

Tags are short textual labels created by users to describe movies.

Examples of tags include:

funny

classic

action

mind-blowing

based on a book

Columns:

COLUMN	DESCRIPTION
userID	User who created the tag
movieID	Movie associated with the tag
Tag	Textual description
Timestamp	Time when the tag was created

2.3.4 WHY MOVIELENS WAS CHOSEN

The MovieLens dataset was selected for the following reasons:

1. It is one of the most widely used benchmark datasets in recommender system research.
2. It contains a large number of ratings, making it suitable for training complex models.
3. It provides additional metadata such as genres and tags, which enable graph-based recommendation experiments.
4. It has high data quality because the ratings were collected through controlled user interfaces.

These characteristics make MovieLens particularly suitable for comparing traditional collaborative filtering models with modern graph neural network approaches.

CHAPTER 3: RELATED WORK

Recommendation systems have become an important research area in machine learning and data science due to their ability to help users discover relevant items from large collections of content. Over the past two decades, many techniques have been proposed for recommendation, ranging from classical collaborative filtering methods to more recent deep learning and graph-based approaches.

3.1. COLLABORATIVE FILTERING APPROACHES

Collaborative filtering is one of the most widely used approaches in recommender systems. It operates under the assumption that users who have exhibited similar preferences in the past are likely to prefer similar items in the future.

Early recommender systems used user-based collaborative filtering, where recommendations were generated by identifying users with similar rating patterns and suggesting items liked by those similar users. However, user-based methods often suffer from scalability problems when the number of users becomes very large.

To address these limitations, item-based collaborative filtering(ItemKNN) was introduced by Badrul Sarwar and colleagues. Their work showed that computing similarities between items instead of users significantly improves scalability and stability in recommendation systems. Item-based collaborative filtering calculates similarity scores between items using historical rating data and recommends items that are similar to those already liked by the user.

This approach became widely adopted in real-world systems because item relationships are generally more stable than user relationships, making recommendations more reliable.

3.2. MATRIX FACTORIZATION METHODS

Another major breakthrough in recommender systems was the introduction of matrix factorization techniques. These methods became particularly popular after the success of collaborative filtering algorithms during the Netflix Prize competition.

Research by Yehuda Koren, Robert Bell, and Chris Volinsky demonstrated that matrix factorization models can effectively capture latent relationships between users and items.

Matrix factorization represents both users and items using latent embedding vectors. These vectors capture hidden characteristics such as user preferences and item attributes that are not explicitly available in the dataset. Recommendations are generated by computing the interaction between user and item embeddings.

Later research introduced Bayesian Personalized Ranking (BPR), proposed by Steffen Rendle. BPR is a pairwise ranking optimization method designed for recommendation tasks where the goal is to rank relevant items higher than irrelevant ones. Instead of predicting exact ratings, BPR focuses on learning user preferences through relative comparisons between positive and negative items.

Matrix factorization with BPR has become a strong baseline in many recommender system studies due to its effectiveness and computational efficiency.

3.3. GRAPH-BASED RECOMMENDATION MODELS

In recent years, graph-based methods have emerged as powerful techniques for recommendation tasks. Graph representations allow complex relationships between users, items, and other entities to be modeled more naturally.

Graph-based recommendation systems represent users and items as nodes in a graph, while interactions such as ratings are represented as edges. This representation enables models to capture higher-order relationships between nodes.

One influential model in this area is GraphSAGE, introduced by William Hamilton and collaborators. GraphSAGE is a graph neural network framework that learns node embeddings by sampling and aggregating information from neighboring nodes. Unlike earlier graph embedding methods, GraphSAGE can generalize to previously unseen nodes and supports heterogeneous graphs containing different node types.

GraphSAGE is particularly useful when recommendation systems need to incorporate additional information such as content features or metadata.

3.4. GRAPH NEURAL NETWORKS FOR RECOMMENDATION

Another important development in graph-based recommendation is the introduction of Light Convolutional Graph Network(LightGCN), proposed by Xiangnan He and colleagues.

LightGCN simplifies traditional graph convolutional networks by removing feature transformation and nonlinear activation functions. Instead, it focuses solely on neighborhood aggregation, which has been shown to be the most important operation for collaborative filtering tasks.

This simplified architecture reduces model complexity and improves training efficiency while still capturing useful collaborative signals from the user–item interaction graph.

LightGCN has demonstrated strong performance in many recommendation benchmarks and is widely considered one of the most effective graph-based collaborative filtering models.

3.5 BENCHMARK DATASETS FOR RECOMMENDATION

Many recommender system studies rely on benchmark datasets for evaluation. One of the most widely used datasets is the MovieLens 20M Dataset, developed by GroupLens Research.

This dataset contains over 20 million movie ratings, along with movie metadata and user-generated tags. Because of its size, quality, and availability of additional features such as genres and tags, MovieLens is widely used for evaluating recommendation algorithms.

3.6 RESEARCH GAP

Although collaborative filtering methods such as ItemKNN and matrix factorization remain strong baselines, they rely mainly on user–item interaction data and may not fully capture richer relationships between entities.

Graph Neural Networks(GNN) provide a promising solution, because they allow recommendation systems to model multiple types of relationships simultaneously. However, there is still ongoing research on how to effectively integrate domain knowledge, such as movie genres and tags, into graph-based recommendation models.

CHAPTER 4: PROPOSED METHODOLOGY

OVERVIEW

The proposed methodology focuses on designing and evaluating a graph-based movie recommendation system with domain specific knowledge that integrates both traditional collaborative filtering techniques and modern graph neural network models. The goal is to compare the effectiveness of classical recommender system approaches with graph-based learning methods that can incorporate richer relationships between users, movies, and domain-specific attributes such as genres and tags.

The methodology consists of several stages including data preprocessing, feature transformation, graph construction, model training, evaluation, and deployment. Each stage ensures that the dataset is prepared appropriately for both baseline recommendation models and graph neural network models.

The overall workflow of the system can be summarized as:

Raw Dataset → Data Preprocessing → Feature Encoding → Graph Construction → Model Training → Model Evaluation → Deployment via Streamlit Dashboard

4.1 DATA PREPROCESSING

The first stage of the methodology involves preparing the MovieLens dataset for analysis and model training. Since raw datasets often contain inconsistencies or missing values, preprocessing ensures data quality and consistency.

The preprocessing pipeline consists of the following steps.

Loading Raw Data

The raw dataset includes three main files:

- ratings.csv
- movies.csv
- tags.csv

These files were loaded into the system using Python data processing libraries. Initial inspection was performed to verify dataset structure, including the number of rows, column names, and data types.

The dataset statistics observed during preprocessing include:

- 20,000,263 rating interactions
- 27,278 movies
- 138,493 users
- 465,564 tag records

Sanity checks were performed to confirm that all required columns such as `userId`, `movieId`, and `rating` were present.

Duplicate Record Verification

Duplicate interactions can introduce bias into recommendation models. Therefore, duplicate user–movie pairs were checked using the (`userId`, `movieId`) combination.

The analysis showed that the dataset contained no duplicate user–movie interactions, meaning that each user rated each movie only once. Since no duplicates were found, no rows were removed in this step.

Handling Missing Values

Missing values were analyzed across the ratings, movies, and tags datasets.

The ratings and movies datasets did not contain missing values in critical columns. However, the tags dataset contained 16 rows with missing tag values. These rows were removed because they did not provide meaningful textual information.

After cleaning, all datasets contained complete and valid records suitable for further processing.

4.2. IDENTIFIER ENCODING

Graph neural networks (GNN) require node identifiers to be represented as continuous integer indices. However, the original MovieLens dataset uses large and non-contiguous identifiers for users and movies.

To address this issue, identifiers were encoded into sequential indices.

User ID Encoding

Each unique `userId` was mapped to a continuous index:

`userId` $\rightarrow$ `user_idx`

This transformation created 138,493 user indices, representing individual users as nodes in the graph.

Movie ID Encoding

Similarly, movie identifiers were mapped to continuous indices:

`movieId` $\rightarrow$ `movie_idx`

This resulted in 26,744 movie nodes, which represent movies that appear in the rating interactions.

Mapping tables were saved to preserve the relationship between original identifiers and encoded indices.

4.3. GENRE PROCESSING

To incorporate domain-specific knowledge into the recommendation system, movie genres were processed as a separate node type.

The genres column in the movies dataset contains multiple genres separated by a vertical bar (|). Each movie can belong to multiple genres.

The preprocessing involved the following steps:

1. Splitting the genre strings using the | delimiter
2. Extracting all unique genre categories
3. Assigning each genre a unique index (genre_idx)
4. Creating edges between movies and their associated genres

This process produced:

- **20 unique genre nodes**
- **53,419 movie–genre relationships**

These relationships enable the graph model to understand semantic similarities between movies.

4.4. GRAPH CONSTRUCTION

After preprocessing and encoding, the dataset was transformed into a graph representation suitable for graph neural networks.

Node Types

The graph includes several types of nodes:

- User nodes
- Movie nodes
- Genre nodes
- Tag nodes

Each node type represents a different entity in the recommendation system.

Edge Types

Edges represent relationships between nodes.

The graph includes the following relationships:

User–Movie edges

These edges represent rating interactions between users and movies.

User → Movie

These edges form the core collaborative filtering signal.

Movie–Genre edges

These edges represent semantic relationships between movies and their genres.

Movie → Genre

These connections allow the model to capture domain-specific movie characteristics.

Movie–Tag edges

Tags created by users were also incorporated as nodes connected to movies.

Movie → Tag

Tags provide additional descriptive information about movies.

4.5. RECOMMENDATION MODELS

To evaluate the effectiveness of graph-based recommendation approaches, four different models were implemented and compared.

Two models represent traditional collaborative filtering baselines, while two models represent graph-based neural network approaches.

4.5.1 BASELINE MODELS

1. ITEM-BASED COLLABORATIVE FILTERING(ItemKNN)

Item-based collaborative filtering recommends movies that are similar to movies previously liked by the user.

The algorithm calculates similarity between items using historical user rating patterns. Cosine similarity was used to measure similarity between movie vectors.

For each movie, the system identifies the top K most similar movies, referred to as the K nearest neighbors.

During recommendation, movies similar to those previously rated highly by the user are suggested.

The best configuration for the model used 50 nearest neighbors.

2. MATRIX FACTORIZATION WITH BAYESIAN PERSONALIZED RANKING(BPR)

Matrix factorization learns latent embeddings representing users and movies in a shared feature space.

Each user and movie is represented by a numerical vector containing hidden preference characteristics.

The model was trained using Bayesian Personalized Ranking (BPR), which optimizes ranking performance by learning to rank positive interactions higher than negative samples.

Training parameters included:

- embedding dimension = 64
- learning rate = 0.001
- training epochs = 5

This approach allows the model to capture hidden preference patterns between users and movies.

4.5.2 GRAPH NEURAL NETWORK MODELS

1. LIGHT CONVOLUTIONAL GRAPH NETWORK (LightGCN)

LightGCN is a simplified graph convolutional network designed specifically for recommendation tasks.

Unlike traditional graph neural networks, LightGCN removes nonlinear transformations and focuses entirely on neighborhood aggregation within the user–item interaction graph.

The model propagates information across the graph to learn user and movie embeddings based on collaborative signals.

Training configuration:

- embedding dimension = 128
- propagation layers = 3
- learning rate = 0.001
- epochs = 15

LightGCN captures higher-order relationships between users and movies by aggregating information from neighboring nodes.

2. GRAPHSAGE

GraphSAGE is a graph neural network architecture that learns node embeddings by sampling and aggregating information from neighboring nodes.

Unlike LightGCN, GraphSAGE can operate on heterogeneous graphs containing multiple node types.

In this project, GraphSAGE incorporates domain knowledge by including genres and tags as additional nodes in the graph.

Training configuration:

- embedding dimension = 128
- hidden dimension = 128
- number of layers = 3
- dropout rate = 0.05
- epochs = 15

This model enables recommendations that consider both collaborative interactions and domain-specific movie attributes.

4.6 MODEL DEPLOYMENT

To demonstrate the functionality of the trained models, an interactive application was developed using Streamlit.

The dashboard allows users to:

- Select a recommendation model
- Input a user ID
- Generate top-K movie recommendations
- Compare results across models

The trained models were hosted and integrated into the dashboard, enabling real-time recommendation generation.

This deployment component provides a practical demonstration of the recommendation system beyond offline evaluation.

4.7. MODEL STORAGE AND REPRODUCIBILITY

To ensure reproducibility and facilitate easy access to the trained recommendation models, all trained models were stored and hosted using the Hugging Face model repository platform.

Hugging Face provides a centralized platform for sharing machine learning models, allowing researchers to publish trained models together with configuration files and training logs. By hosting the trained models online, the recommendation system becomes easier to reproduce, share, and deploy.

In this project, the following trained models were uploaded and stored in the Hugging Face repository:

- Item-Based Collaborative Filtering (ItemKNN)
- Matrix Factorization with Bayesian Personalized Ranking (MF-BPR)
- LightGCN
- GraphSAGE

Each repository includes the trained model weights, configuration files, and training logs required to reproduce the experiments.

The stored models allow external users to download the trained checkpoints and generate recommendations without retraining the models from scratch. This approach improves experiment reproducibility and supports future extensions of the recommendation system.

The hosted models are also integrated into the Streamlit demonstration application, where they are loaded dynamically to generate real-time recommendations for selected users.

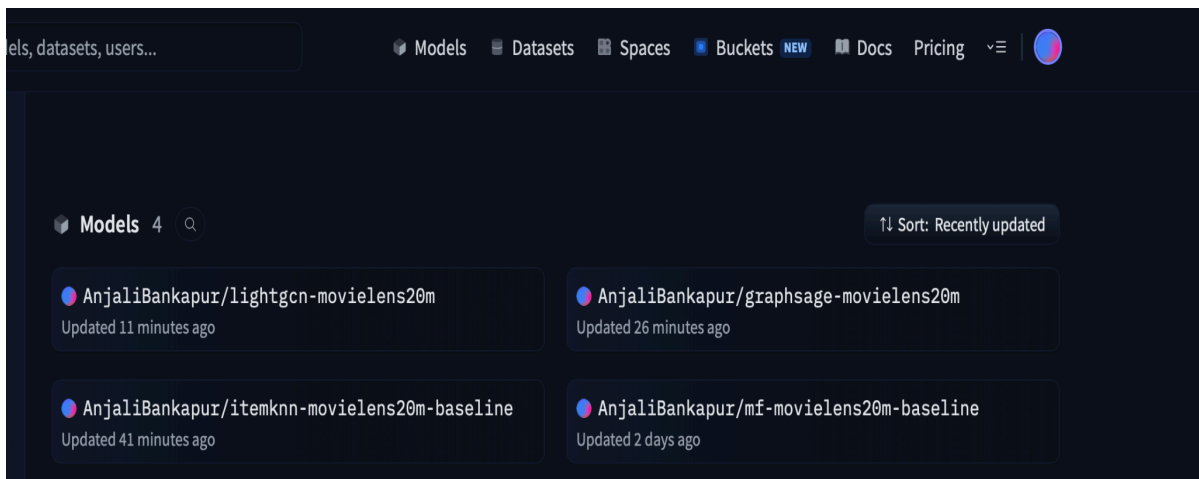


FIGURE: 4 MODELS STORED MAIN PAGE

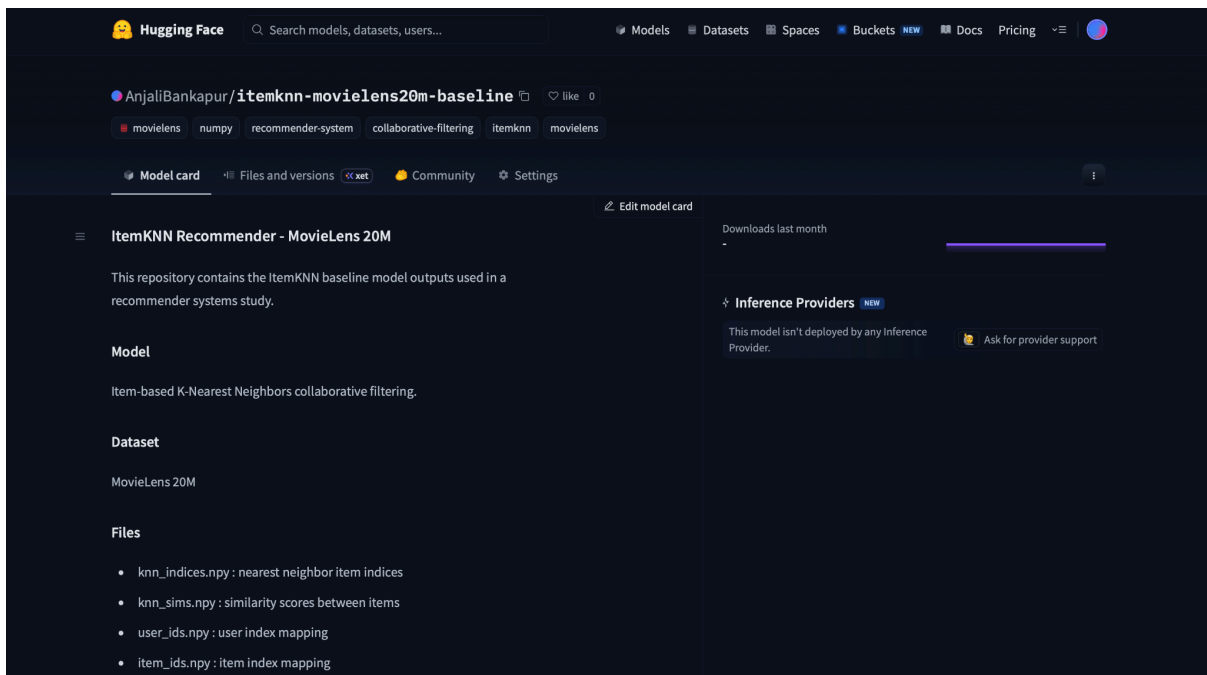


FIGURE: ITEMKNN MODEL STORAGE

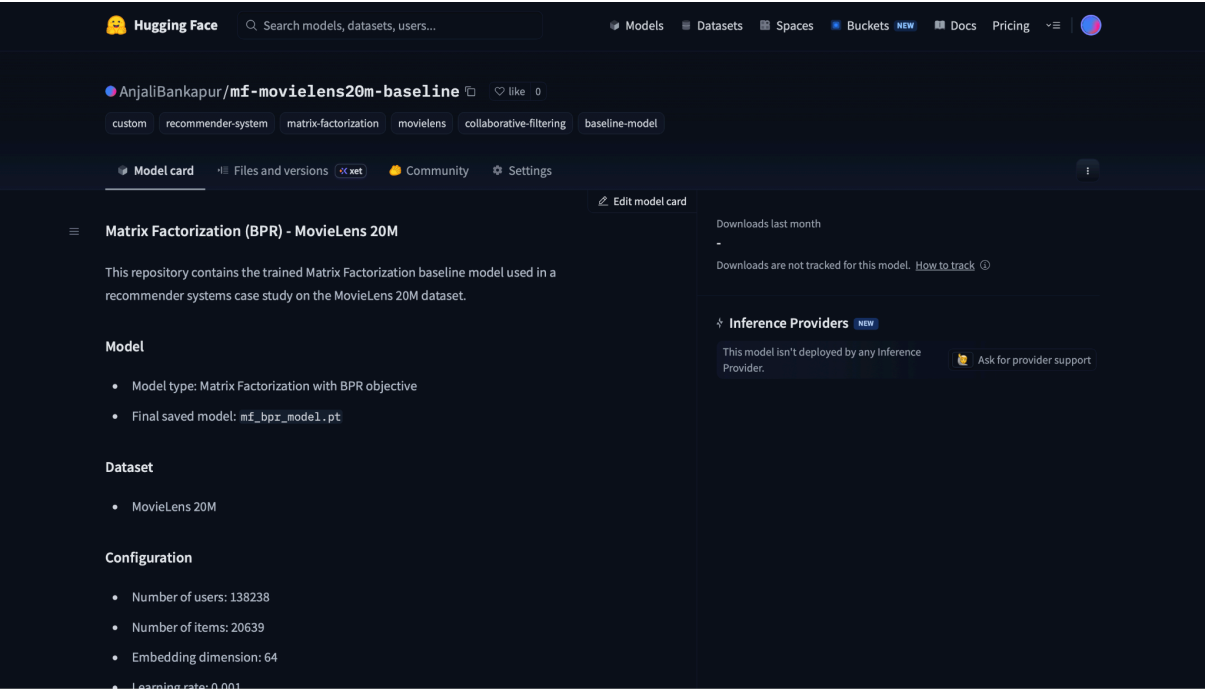


FIGURE: MF-BPR MODEL STORAGE

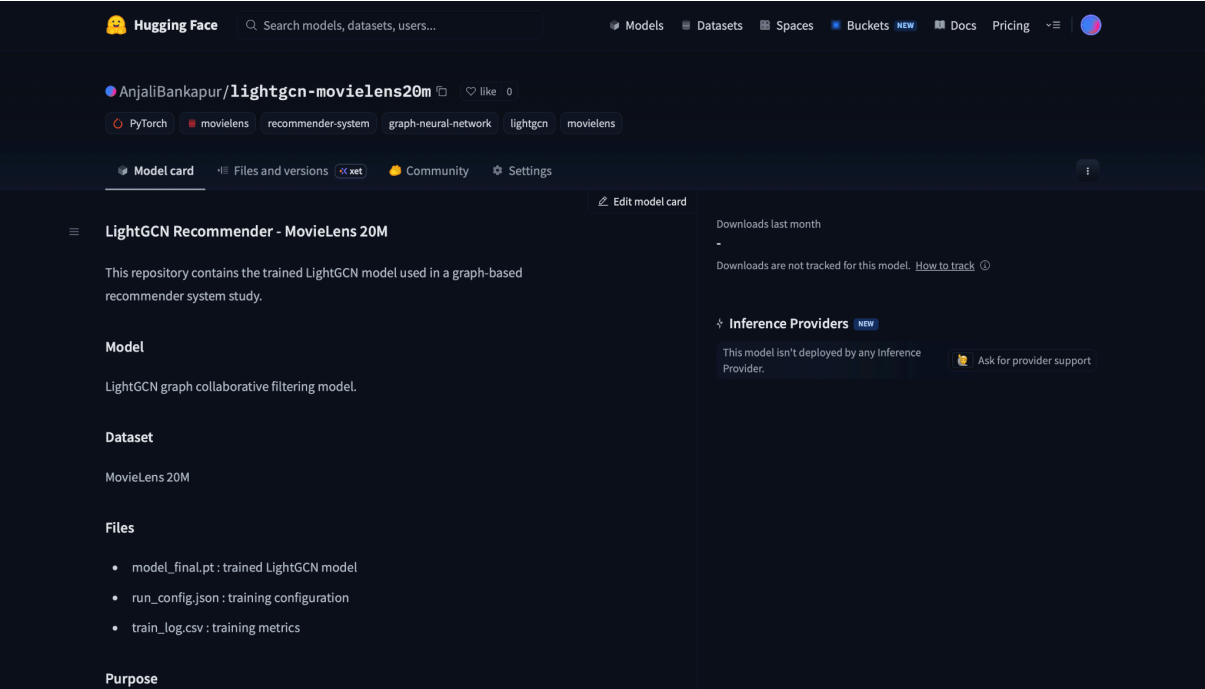


FIGURE: LIGHTGCN MODEL STORAGE

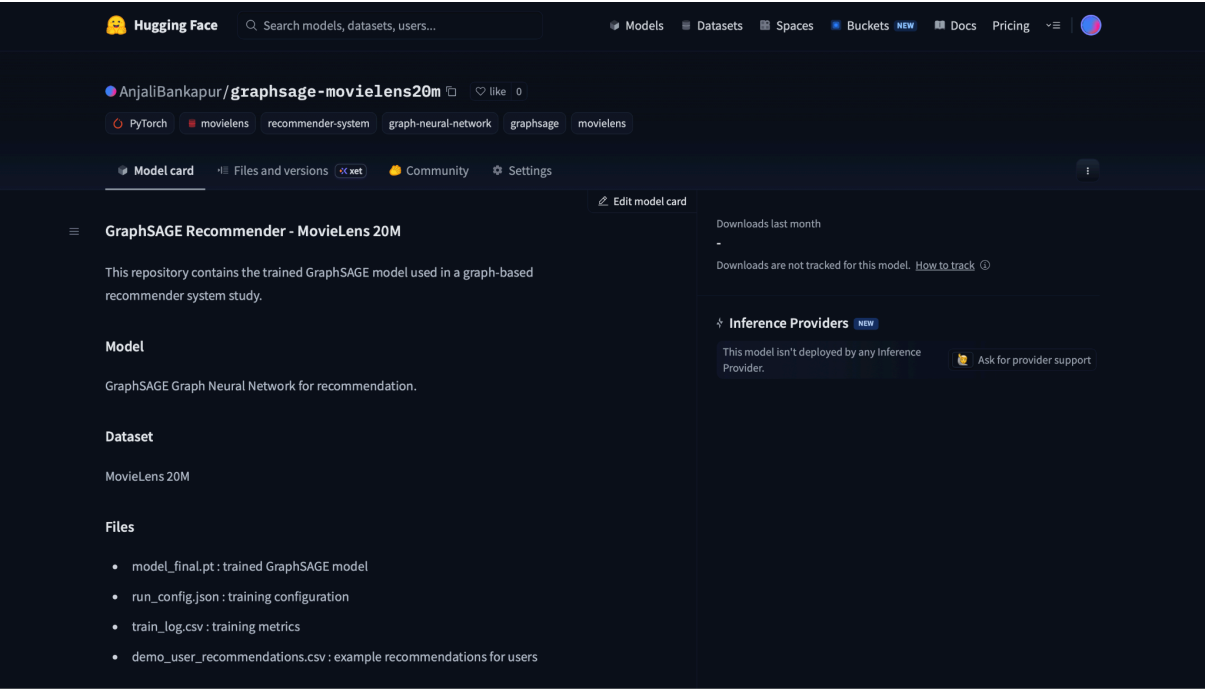


FIGURE: GRAPHSAGE MODEL STORAGE

4.8 SUMMARY OF THE METHODOLOGY

The proposed methodology integrates data preprocessing, graph construction, multiple recommendation models, and evaluation into a unified pipeline.

By comparing traditional collaborative filtering methods with graph neural network approaches, the study provides insights into the strengths and limitations of each technique.

Furthermore, the integration of domain-specific knowledge through genres and tags allows the graph-based models to capture richer relationships between movies, offering a more expressive representation of recommendation data.

CHAPTER 5: EVALUATION

5.1. EVALUATION OVERVIEW

The goal of a recommender system is not only to predict ratings accurately but to rank items in a way that the most relevant items appear in the top positions. Therefore, ranking-based metrics provide a more realistic assessment of recommendation quality.

In this project, four models were evaluated:

- Item-Based Collaborative Filtering (ItemKNN)
- Matrix Factorization with Bayesian Personalized Ranking (MF-BPR)
- LightGCN
- GraphSAGE

All models were evaluated using the same testing protocol to ensure a fair comparison.

5.2. DATA SPLITTING STRATEGY

To evaluate recommendation performance, the dataset was divided into training, validation, and testing sets.

The splitting strategy followed a leave-last-interaction-out approach, which is widely used in recommendation evaluation.

For each user:

- Earlier interactions were used for training.
- The most recent interaction was used for testing.

This ensures that the model predicts future user preferences based on past behavior.

For the graph-based models, an additional validation split was used.

DATASET SPLIT SUMMARY:

DATASET SPLIT	NUMBER OF INTERACTIONS
Training Set	19,723,277
Validation Set	138,493
Test Set	138,493

This approach guarantees that each user appears in the training set and has one held-out interaction for evaluation.

5.3. NEGATIVE SAMPLING STRATEGY

In real-world recommendation systems, users interact with only a small fraction of available items. Therefore, evaluation must consider both relevant and non-relevant items.

To simulate a realistic recommendation scenario, negative sampling was applied during evaluation.

For each user:

1. The true positive movie from the test set was selected.
2. 499 negative movies were randomly sampled from movies that the user had not interacted with.
3. The model ranked all 500 movies (1 positive + 499 negatives).

The ranking position of the positive movie within this list was used to compute evaluation metrics.

This approach ensures that the evaluation measures how well the model distinguishes relevant items from irrelevant ones.

5.4. EVALUATION METRICS

Four ranking-based metrics were used to evaluate recommendation performance.

Metrics were computed at two cutoff values:

- $K = 10$
- $K = 20$

These values correspond to the number of recommendations shown to the user.

Precision@K

Precision@K measures the proportion of recommended items that are relevant.

The formula is

$$\text{Precision@K} = (\text{Number of relevant items in top K}) / K$$

In this evaluation setting, there is only one relevant item in the candidate list. Therefore, Precision@K measures whether the correct movie appears among the top-K recommendations.

Higher precision indicates that the model recommends relevant movies more accurately.

Recall@K

Recall@K measures how many relevant items were successfully retrieved among the top-K recommendations.

The formula is:

$$\text{Recall@K} = (\text{Number of relevant items retrieved}) / (\text{Total relevant items})$$

Since each user has one relevant item in the evaluation set, recall indicates whether that item appears within the top-K results.

Higher recall indicates that the system is better at retrieving relevant items.

NDCG@K (Normalized Discounted Cumulative Gain)

NDCG evaluates the ranking quality of the recommendation list.

Unlike Precision and Recall, NDCG considers the position of the relevant item.

Relevant items appearing at higher positions receive higher scores.

The general formula is:

$$\text{NDCG@K} = \text{DCG@K} / \text{IDCG@K where:}$$

- DCG measures the discounted gain based on ranking position
- IDCG represents the ideal ranking score

Higher NDCG values indicate that relevant items appear closer to the top of the recommendation list.

HitRate@K

Hit Rate measures whether the correct item appears within the top-K recommendations.

The formula is

HitRate@K = 1 if the relevant item appears in top K

HitRate@K = 0 otherwise

The average Hit Rate across all users indicates how frequently the model successfully recommends the correct item.

This metric is often used in recommendation systems because it reflects whether the user would see the relevant item in the recommendation list.

5.4. EXPERIMENTAL RESULTS

The performance of the four models is summarized in the following table.

MODEL S	PRECISION@10	RECALL@10	NDCG@10	HITRATE@10	PRECISION@20	RECALL@20	NDCG@20	HITRATE@20
ITEMKN N	0.0682	0.6817	0.4166	0.6817	0.0401	0.8010	0.8010	0.8010
MF-BPR	0.0575	0.5751	0.3397	0.5751	0.0365	0.7300	0.3397	0.7300
LIGHTG CN	0.0489	0.4888	0.2810	0.4888	0.0323	0.6462	0.2810	0.6462
GRAPHS AGE	0.0483	0.4888	0.2810	0.4888	0.0303	0.6062	0.2010	0.62622

5.5. RESULTS ANALYSIS

The experimental results reveal several important observations.

ITEM-BASED COLLABORATIVE FILTERING

ItemKNN achieved the strongest performance across all evaluation metrics. This indicates that item similarity patterns in the MovieLens dataset provide strong signals for recommendation. Since many users rate popular movies, item-based similarity can effectively capture user preferences.

MATRIX FACTORIZATION WITH BAYESIAN PERSONALIZED RANKING (BPR)

Matrix Factorization performed slightly lower than ItemKNN but still produced strong results. By learning latent embeddings, MF captures hidden relationships between users and movies that are not directly observable from rating data.

LIGHT CONVOLUTIONAL GRAPH NETWORK (LIGHTGCN)

LightGCN showed moderate performance improvements compared to basic collaborative filtering models in some studies, but in this experiment it did not outperform classical baselines. While it captures higher-order collaborative relationships through graph propagation, the simpler structure of the dataset may favor traditional methods.

GRAPHSAGE

GraphSAGE achieved the lowest ranking metrics among the four models. This is likely due to the increased complexity of the heterogeneous graph and the relatively sparse nature of metadata such as tags.

However, GraphSAGE provides a richer modeling framework by incorporating domain knowledge such as genres and tags. This allows the system to represent more complex relationships between movies and user preferences.

5.6. EVALUATION SUMMARY

The evaluation demonstrates that classical collaborative filtering methods remain highly competitive on dense recommendation datasets such as MovieLens 20M. Item-based collaborative filtering achieved the best performance, followed by matrix factorization.

Graph-based models provide additional flexibility and the ability to incorporate semantic relationships. However, their advantages may become more apparent in datasets where content features and complex relationships play a larger role.

Overall, the evaluation highlights the importance of comparing new models against strong baseline methods to ensure a fair and meaningful performance assessment.

CHAPTER 6: RESULTS

6.1. OVERVIEW OF EXPERIMENTAL RESULTS

After training the recommendation models, their performance was evaluated using ranking-based evaluation metrics. These metrics measure how effectively the models rank relevant movies near the top of the recommendation list.

The models evaluated in this study include:

- Item-Based Collaborative Filtering (ItemKNN)
- Matrix Factorization with Bayesian Personalized Ranking (MF-BPR)
- LightGCN
- GraphSAGE

Each model generated ranked recommendation lists for users, and performance was measured using the following metrics:

- Precision@10 and Precision@20
- Recall@10 and Recall@20
- NDCG@10 and NDCG@20
- HitRate@10 and HitRate@20

These metrics were computed using the evaluation procedure described earlier, where each user's test interaction was ranked among 499 negative candidate items.

6.2. QUANTITATIVE RESULTS

The performance results for the four models are summarized in Table 1.

MODEL S	PRECISIO N@10	RECALL @10	NDCG@ 10	HITRA TE@10	PRECI SION@ 20	RECAL L@20	NDCG @20	HITR ATE @20
ITEMKN N	0.0682	0.6817	0.4166	0.6817	0.0682	0.8010	0.8010	0.8010
MF-BPR	0.0575	0.5751	0.3397	0.5751	0.0365	0.7300	0.3397	0.7300
LIGHTG CN	0.0489	0.4888	0.2810	0.4888	0.0323	0.6462	0.2810	0.6462
GRAPHS AGE	0.0483	0.4888	0.2810	0.4888	0.0303	0.6062	0.2010	0.6262 2

TABLE 1: RECOMMENDATION MODEL PERFORMANCE COMPARISION

6.3. PERFORMANCE ANALYSIS

The experimental results show noticeable differences in performance between the baseline collaborative filtering models and the graph neural network models.

ITEM-BASED COLLABORATIVE FILTERING (ITEMKNN)

ItemKNN achieved the best performance across all evaluation metrics. In particular, it achieved the highest values for Precision@10, Recall@10, and NDCG@10.

This result suggests that item similarity patterns in the MovieLens dataset are strong indicators of user preferences. Since many users tend to watch and rate similar groups of movies, item-based similarity provides an effective mechanism for generating accurate recommendations.

ItemKNN also achieved the highest Recall@20 and HitRate@20, indicating that it successfully retrieved relevant movies within the top recommendation positions for a large proportion of users.

The ItemKNN model achieved:

- Precision@10 = 0.0682
- Recall@10 = 0.6817
- NDCG@10 = 0.4166
- HitRate@10 = 0.6817
- Precision@20 = 0.0682
- Recall@20 = 0.8010
- NDCG@20 = 0.8010
- HitRate@20 = 0.8010

Matrix Factorization (MF-BPR)

Matrix Factorization produced the second-best performance across most evaluation metrics. This method learns latent embeddings representing users and movies in a shared feature space.

The MF-BPR model achieved:

- $\text{Precision@10} = 0.0575$
- $\text{Recall@10} = 0.5751$
- $\text{NDCG@10} = 0.3397$
- $\text{HitRate@10} = 0.5751$
- $\text{Precision@20} = 0.0365$
- $\text{Recall@20} = 0.7300$
- $\text{NDCG@20} = 0.3397$
- $\text{HitRate@20} = 0.7300$

Although these values are slightly lower than those achieved by ItemKNN, the model still demonstrates strong recommendation capability.

Matrix factorization captures hidden preference patterns that are not directly observable in rating data, which helps the model generalize to new recommendation scenarios.

LightGCN

LightGCN achieved moderate performance compared with the baseline models. Although LightGCN is designed specifically for collaborative filtering tasks, its performance in this experiment did not surpass the simpler baseline approaches.

The LightGCN model achieved:

- $\text{Precision@10} = 0.0489$

- $\text{Recall@10} = 0.4888$
- $\text{NDCG@10} = 0.2810$
- $\text{HitRate@10} = 0.4888$
- $\text{Precision@20} = 0.0323$
- $\text{Recall@20} = 0.6462$
- $\text{NDCG@20} = 0.2810$
- $\text{HitRate@20} = 0.6462$

These results indicate that the model was able to learn useful collaborative relationships within the user–movie interaction graph, but the simpler baseline models still produced stronger ranking results.

One possible explanation is that the MovieLens dataset already contains strong item similarity patterns, which can be captured effectively by traditional collaborative filtering techniques.

GraphSAGE

GraphSAGE produced the lowest ranking metrics among the four models.

The model achieved:

- $\text{Precision@10} = 0.0483$
- $\text{Recall@10} = 0.4888$
- $\text{NDCG@10} = 0.2810$
- $\text{HitRate@10} = 0.4888$
- $\text{Precision@20} = 0.0303$
- $\text{Recall@20} = 0.6062$
- $\text{NDCG@20} = 0.2010$
- $\text{HitRate@20} = 0.62622$

Although GraphSAGE did not outperform the other models in terms of ranking accuracy, it has an important advantage: it incorporates domain-specific knowledge through genres and tags.

Unlike the other models, GraphSAGE operates on a heterogeneous graph that includes multiple node types such as users, movies, genres, and tags. This allows the system to capture richer relationships between entities.

However, the additional complexity of the graph may also introduce noise and make optimization more challenging during training.

6.4. COMPARISON BETWEEN BASELINE AND GRAPH MODELS

The results reveal several interesting observations when comparing baseline models with graph neural network models.

First, classical collaborative filtering approaches remain highly competitive. Both ItemKNN and Matrix Factorization achieved higher ranking performance than the graph-based models in this study.

Second, graph-based models provide a more flexible representation of recommendation data. LightGCN captures higher-order relationships between users and movies, while GraphSAGE allows additional domain-specific features such as genres and tags to be incorporated.

Third, the inclusion of additional metadata does not always guarantee improved ranking performance. In some cases, the additional complexity may introduce noise or make training more difficult.

6.5. IMPACT OF RECOMMENDATION LIST SIZE

The evaluation metrics were computed at two recommendation list sizes: Top-10 and Top-20.

Increasing the recommendation list size generally improves recall and hit rate, since the system has more opportunities to include the correct item.

For example:

- Recall@10 for ItemKNN = 0.6817
- Recall@20 for ItemKNN = 0.8010

This indicates that approximately 80% of relevant items appear within the top 20 recommendations.

However, increasing the recommendation list size typically reduces precision slightly, since the additional recommendations may include less relevant items.

6.6. KEY FINDINGS

The results of this experiment highlight several key findings:

1. Item-based collaborative filtering achieved the strongest performance on the MovieLens dataset.
2. Matrix factorization remains a powerful baseline method capable of capturing hidden user preference patterns.
3. Graph neural networks provide a richer representation of recommendation data but may require additional tuning to outperform classical methods.
4. Incorporating domain-specific knowledge through genres and tags improves the expressiveness of the model but does not always lead to higher ranking accuracy.

6.7. SYSTEM DEMONSTRATION USING STREAMLIT

To demonstrate the practical functionality of the trained recommendation models, an interactive web application was developed using the Streamlit.

The Streamlit application provides a user-friendly interface that allows real-time interaction with the trained recommendation models. This interface enables users to generate personalized movie recommendations and compare the outputs of different recommendation algorithms.

6.7.1 PURPOSE OF THE STREAMLIT APPLICATION

The Streamlit dashboard was developed for the following purposes:

- To provide an interactive demonstration of the trained recommendation models
- To allow real-time recommendation generation
- To compare results across different models
- To visualize the performance and behavior of the recommendation system

This application serves as a practical demonstration of how the recommendation system can be deployed in a real-world environment.

6.7.2 DASHBOARD FEATURES

The Streamlit application includes multiple components that allow users to explore the recommendation system.

MODEL SELECTION

Users can select one of the four trained models:

- Item-Based Collaborative Filtering (ItemKNN)
- Matrix Factorization (MF-BPR)

- LightGCN
- GraphSAGE

This enables direct comparison of recommendations produced by different algorithms.

USER RECOMMENDATION INPUT

The dashboard allows users to enter a user ID from the dataset.

After entering a user ID, the system generates Top-K movie recommendations for that specific user based on the selected model.

TOP-K RECOMMENDATION OUTPUT

The system displays a ranked list of recommended movies for the selected user. Each recommendation includes:

- Rank
- MovieID
- Movie Title
- Movie Genres
- Tag metadata

This allows users to understand the type of movies suggested by the recommendation model.

MODEL COMPARISON

The interface allows switching between different recommendation models to observe differences in recommendations.

This functionality demonstrates how different algorithms may produce different recommendation lists based on their underlying learning mechanisms.

6..7.3. EXAMPLE DEMONSTRATION SCENARIO

For example, when a Specific user ID is entered into the dashboard, the system generates the top recommended movies for that user.

The recommendations differ depending on the selected model:

- ItemKNN recommends movies that are similar to previously liked movies.
- Matrix Factorization (MF-BPR) recommends movies based on learned latent user preferences.
- LightGCN recommends movies based on collaborative graph relationships.
- GraphSAGE recommends movies by considering both collaborative interactions and domain-specific information such as genres and tags.

This interactive comparison helps illustrate the strengths and behavior of different recommendation approaches.

MODEL STORAGE:

The trained models were hosted using Hugging Face repositories to ensure reproducibility and enable easy integration with the Streamlit demonstration application.

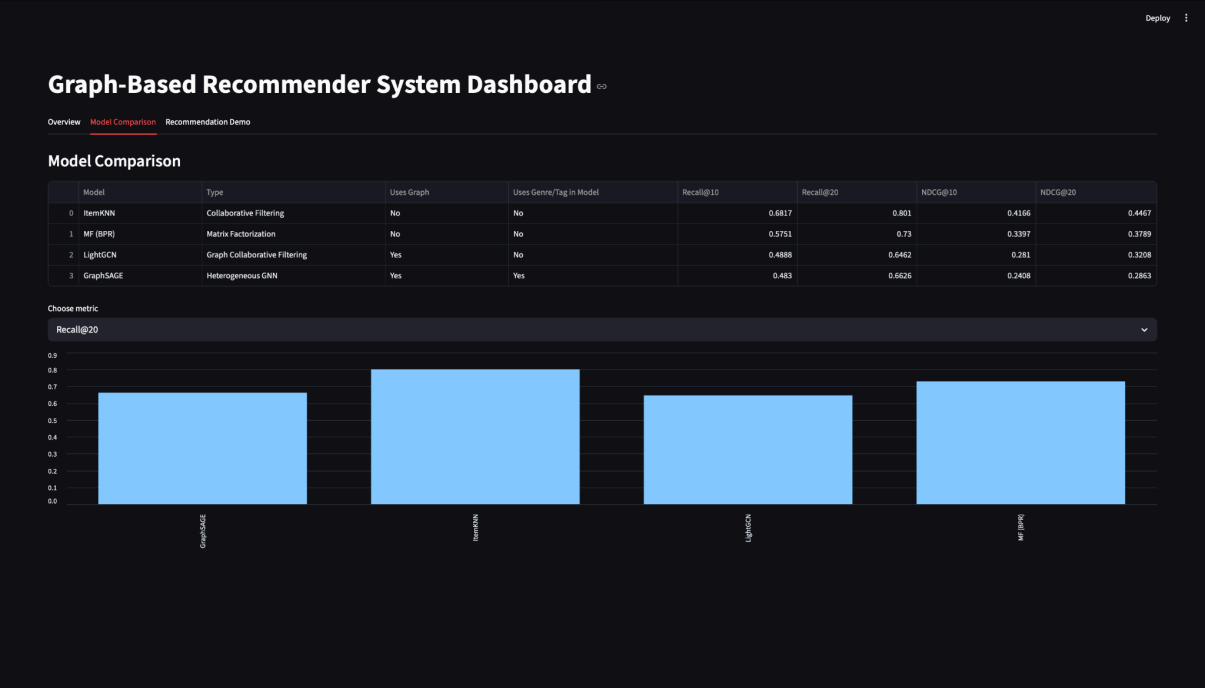


FIGURE 1: STREAMLIT MODEL COMPARISON RESULTS

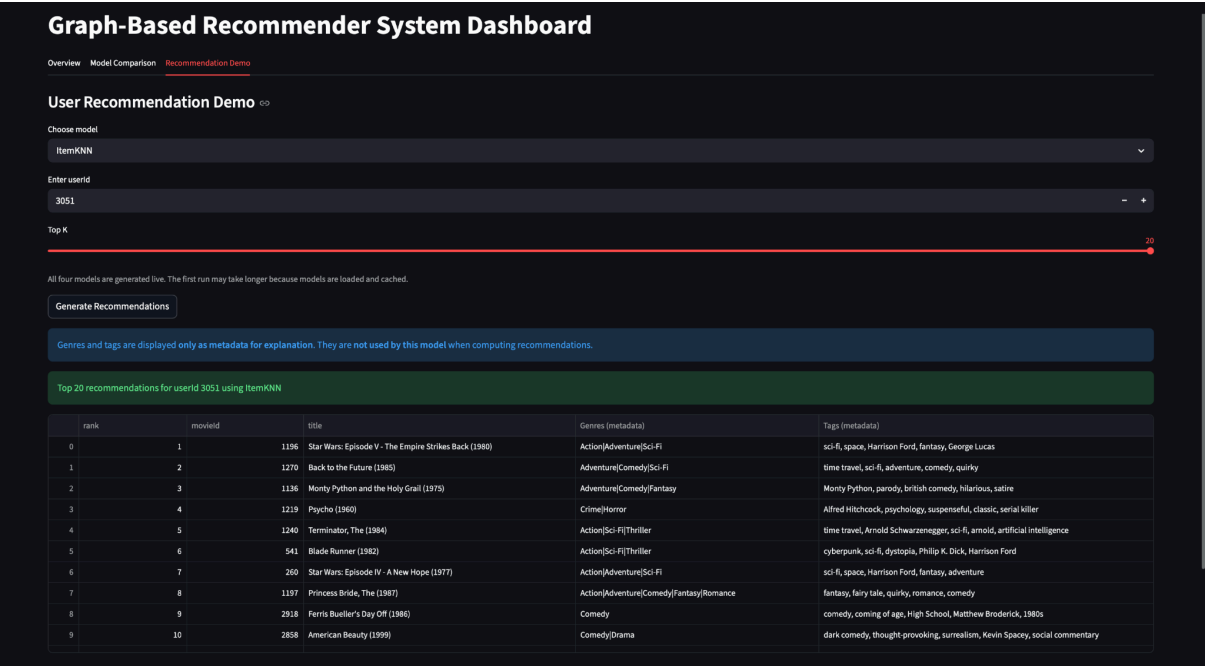


FIGURE 2: TOP-10/20 MOVIE RECOMMENDATION FOR DIFFERENT MODELS

IMPORTANCE OF THE DEMONSTRATION

The Streamlit shows the model comparison and top 10/20 movies. And there are two dropdowns: one is to choose a model, in that one can select the 4 different models like ItemKNN, Matrix-Factorization (MF-BPR), LightGCN, GraphSAGE. And, the second one Enter userID, in that one can select different userID and finally, after clicking on the option called enter, after that we can see the list of movie recommendations of top10/20.

CHAPTER 7: DISCUSSION

7.1. OVERVIEW

The objective of this study was to investigate the effectiveness of both traditional collaborative filtering techniques and graph-based recommendation models for movie recommendation tasks. Four models were implemented and evaluated: Item-Based Collaborative Filtering (ItemKNN), Matrix Factorization with Bayesian Personalized Ranking (MF-BPR), LightGCN, and GraphSAGE.

The results show that classical collaborative filtering methods remain strong baselines for recommendation tasks, particularly when the dataset contains a large number of user–item interactions. At the same time, graph neural network models provide a more flexible framework capable of incorporating richer relationships between entities such as movies, genres, and tags.

This section discusses the key insights obtained from the experimental results.

7.2. PERFORMANCE OF BASELINE MODELS

The results indicate that Item-Based Collaborative Filtering (ItemKNN) achieved the best performance across all ranking metrics. This outcome suggests that the MovieLens dataset contains strong item similarity patterns. When users tend to watch and rate similar groups of movies, item-based similarity methods can effectively identify related items and generate accurate recommendations.

ItemKNN benefits from the fact that movie relationships are relatively stable. If many users who liked one movie also liked another movie, the algorithm can

easily detect this similarity and recommend those items to other users with similar preferences.

Matrix Factorization (MF-BPR) also demonstrated strong performance. By learning latent embeddings for users and movies, the model captures hidden preference patterns that are not directly observable in the dataset. This allows the system to generalize recommendations beyond simple item similarity.

Although Matrix Factorization did not outperform ItemKNN in this experiment, it remains a powerful approach because it models user preferences in a compact latent feature space.

7.3. PERFORMANCE OF GRAPH-BASED MODELS

The graph-based models, LightGCN and GraphSAGE, produced lower ranking metrics compared with the baseline methods in this study. However, this result does not necessarily imply that graph-based methods are inferior. Instead, it highlights the importance of understanding the structure and characteristics of the dataset.

LightGCN operates on a collaborative graph consisting of users and movies connected through interaction edges. By propagating information across the graph, the model captures higher-order relationships between users and items. This enables the system to learn more complex collaborative patterns.

However, the MovieLens dataset already contains strong collaborative signals. In such scenarios, simpler methods such as ItemKNN can perform very well because item similarity patterns are highly informative.

GraphSAGE produced the lowest ranking metrics among the four models. One possible explanation is the increased complexity of the heterogeneous graph used in this model. GraphSAGE incorporates additional node types such as

genres and tags, which introduce additional information but also increase the difficulty of training the model.

Metadata such as tags can also be noisy or sparsely distributed across movies. As a result, the additional domain information may not always provide a strong signal for ranking accuracy.

CHAPTER 8: CONCLUSION

This study developed and evaluated a movie recommendation system by comparing traditional collaborative filtering methods with graph neural network approaches using the MovieLens 20M Dataset. The system incorporated four models: Item-Based Collaborative Filtering (ItemKNN), Matrix Factorization with Bayesian Personalized Ranking (MF-BPR), LightGCN, and GraphSAGE. Experimental results showed that ItemKNN achieved the highest ranking performance, followed by MF-BPR, while the graph-based models provided a more expressive representation of relationships between users, movies, genres, and tags. Although GraphSAGE produced slightly lower ranking metrics, it demonstrated the potential of incorporating domain-specific knowledge into recommendation graphs. The trained models were further deployed through an interactive Streamlit dashboard, enabling real-time recommendation generation and model comparison. Overall, the study highlights that classical collaborative filtering methods remain strong baselines, while graph-based approaches offer a flexible framework for integrating richer relational information in recommendation systems.

CHAPTER 9: REFERENCES

1. Harper, F. M., & Konstan, J. A. (2015).
The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4), 1–19.
This paper describes the MovieLens datasets and their importance for recommender system research.
2. Koren, Y., Bell, R., & Volinsky, C. (2009).
Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8), 30–37.
This study explains how matrix factorization learns latent user and item embeddings for recommendation.
3. Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009).
BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*.
This paper introduced the Bayesian Personalized Ranking (BPR) optimization method for ranking-based recommender systems.
4. Hamilton, W., Ying, R., & Leskovec, J. (2017).
Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems (NeurIPS)*.
This work introduced GraphSAGE, a graph neural network model that learns node embeddings by aggregating information from neighbors.
5. He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020).
LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *Proceedings of the 43rd International ACM SIGIR*

Conference.

This research proposed LightGCN, a simplified graph convolution model designed specifically for collaborative filtering tasks.

6. Paszke, A., Gross, S., Massa, F., et al. (2019).

PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems.

PyTorch was used as the main deep learning framework for implementing graph neural network models.

7. Streamlit Inc. (2023).

Streamlit: The Fastest Way to Build Data Apps in Python.

Streamlit was used to develop the interactive dashboard for real-time recommendation demonstration.